CODES

https://github.com/VikasBhaskar-gif/LRC-HardwareHustle

src/main.cpp

```
#include <Arduino.h>
#include "pump_controller.h"
#include "pump2_controller.h"
#include "pump3_controller.h"
#include "case1_controller.h"
#include "float_switch.h"
#include "flow_sensor.h"
#include "system_switch.h"
#include "start_button.h"
#include "solenoid_valve.h"
#include "commissioning.h"
#include "duty_parser.h"

static bool s_csvLoggingActive = false;
static unsigned long s_lastCsvLogTime = 0;

static bool checkPumpSafety(int pump) {
    if (isCase1EmergencyStopActive()) {
        Serial.printf("[SAFETY] Blocked: Emergency Stop is active! Press
            '%s' or 'n' to clear.\n",
                    (pump == 1) ? "e" : "q");
        return false;
    }
    if (!isSystemOn()) {
        Serial.println("[SAFETY] Blocked: Cannot run pump while System
            Switch is OFF.");
        return false;
    }
    return true;
}

static void printHelpMenu() {
    Serial.println("\n----------------- COMMAND MENU -----------------");
    Serial.println("Auto Sequence: Start Button (GPIO 34) starts Case 1 ->
        Case 2");
    Serial.println("               Case 1: Normal recirculation (P1 50%, P2
        100%, Solenoid OPEN)");
    Serial.println("               Sump High: P1 halts (0%), P2 continues
        100%");
    Serial.println("               Return Normal: 10s delay then Case 2
        auto-starts");
    Serial.println("               Case 2: 90s flush (P1 50%, P2 100%, P3
        50%, Solenoid CLOSED)");
    Serial.println("               Final: 90s timeout -> latched shutdown
        (all motors OFF)");
    Serial.println("System:        s/[Space]=E-Stop | p=Status | h=Help");
    Serial.println("               y=Query System State | Y=Toggle System
        Enable (On/Off)");
    Serial.println("               Commands: sys on / sys off");
    Serial.println("P1 (U2):       1=25% 2=50% 3=75% 4=100% 0=Stop |
        e=Enable d=Disable");
    Serial.println("               d1 <0-100> in 5% steps (e.g. d1 50)");
    Serial.println("P2 (U3):       5=25% 6=50% 7=75% 8=100% 9=Stop |
        q=Enable w=Disable");
    Serial.println("               d<0-100> in 5% steps (e.g. d5, d10, d50,
        d100) | d2 <0-100>");
    Serial.println("               f=Fwd v=Rev o=P2 Status");
    Serial.println("P3 (U4):       u=Start Req (Rejected: Unsafe) |
        i=Stop");
    Serial.println("Solenoid:      a=Open c=Close k=Toggle");
    Serial.println("Sensors:       t=Float Switch | l=Flow Rate | L=CSV
        Log");
    Serial.println("Manual Flow:   N=Manual Start Case 1 (Commissioning)");
    Serial.println("Case 1 Sim:    n=Normal x=Sump High r=Return to Normal
        (Auto Case 2 in 10s)");
    Serial.println("Commission:    T/m=Enter Commissioning Mode");
    Serial.println("------------------------------------------------\n");
}

static void handleSerialCommand(char cmd) {
    switch (cmd) {
        case 's':
        case 'S':
        case ' ': // Spacebar - System Emergency Stop
            case1TriggerEmergencyStop();
            break;

        case 'p':
        case 'P':
            printCase1Status();
            printSystemSwitchStatus();
            printFloatSwitchStatus();
            printFlowSensorStatus();
            printSolenoidStatus();
            printPump1Status();
            printPump2Status();
            printPump3Status();
            printStartButtonStatus();
            break;

        case 'h':
        case 'H':
        case '?':
            printHelpMenu();
            break;

        case 'y':
            Serial.printf("SYSTEM -> %s\n", isSystemOn() ? "ON" : "OFF");
            break;

        case 'Y':
            toggleSystemOn();
            updateCase1WithInputs(isSystemOn(), isFloatSwitchTriggered());
            break;

        case 't':
            Serial.printf("FLOAT -> %s\n", isFloatSwitchTriggered() ? "SUMP
                HIGH" : "NORMAL");
            break;

        case 'T':
        case 'm':
        case 'M':
            enterCommissioningMode();
            break;

        case 'l':
            Serial.printf("FLOW -> %.2f L/min | %.0f Hz\n",
                getFlowRateLPM(), getFlowFrequencyHz());
            break;

        case 'L':
            s_csvLoggingActive = !s_csvLoggingActive;
            if (s_csvLoggingActive) {
                s_lastCsvLogTime = millis();
                Serial.println("\n[LOG] Flow logging started");
Serial.println("timestamp_ms,pump2_pwm,frequency_hz,estimated_flow_lpm");
            } else {
                Serial.println("[LOG] Flow logging stopped\n");
            }
            break;

        case 'a':
        case 'A':
            if (isCase1EmergencyStopActive()) {
                Serial.println("[SAFETY] Blocked: Cannot open solenoid while
                    Emergency Stop is active.");
                break;
            }
            if (!isSystemOn()) {
                Serial.println("[SAFETY] Blocked: Cannot open solenoid while
                    System Switch is OFF.");
                break;
            }
            openSolenoid();
            break;

        case 'c':
        case 'C':
            closeSolenoid();
            break;

        case 'k':
        case 'K':
            if (!isSolenoidOpen()) {
                if (isCase1EmergencyStopActive()) {
                    Serial.println("[SAFETY] Blocked: Cannot open solenoid
                        while Emergency Stop is active.");
                    break;
                }
                if (!isSystemOn()) {
                    Serial.println("[SAFETY] Blocked: Cannot open solenoid
                        while System Switch is OFF.");
                    break;
                }
            }
            toggleSolenoid();
            break;

        case 'n':
            if (isCase1EmergencyStopActive()) {
                case1ClearEmergencyStop();
            }
            case1TriggerSimulatedNormal();
            break;

        case 'N':
            case1StartNormalFlow();
            break;

        case 'x':
        case 'X':
            case1TriggerSimulatedSumpHigh();
            break;

        case 'r':
        case 'R':
            case1TriggerSimulatedReturnToNormal();
            break;

        case '0':
            setPump1Duty(0);
            break;
        case '1':
```

```
case '2':
case '3':
case '4': {
    if (!checkPumpSafety(1)) break;
    uint8_t duty = (cmd == '1') ? 25 : (cmd == '2') ? 50 : (cmd ==
        '3') ? 75 : 100;
    setPump1Duty(duty);
    break;
}
case 'e':
case 'E':
    if (isCase1EmergencyStopActive()) {
        case1ClearEmergencyStop();
    }
    if (!isSystemOn()) {
        Serial.println("[SAFETY] Blocked: Cannot enable driver while
            System Switch is OFF.");
        break;
    }
    enablePump1();
    break;
case 'd':
case 'D':
    disablePump1();
    break;

case '5':
case '6':
case '7':
case '8': {
    if (!checkPumpSafety(2)) break;
    uint8_t duty = (cmd == '5') ? 25 : (cmd == '6') ? 50 : (cmd ==
        '7') ? 75 : 100;
    setPump2Duty(duty);
    break;
}
case '9':
    setPump2Duty(0);
    break;
case 'q':
case 'Q':
    if (isCase1EmergencyStopActive()) {
        case1ClearEmergencyStop();
    }
    if (!isSystemOn()) {
        Serial.println("[SAFETY] Blocked: Cannot enable driver while
            System Switch is OFF.");
        break;
    }
    enablePump2();
    break;
case 'w':
case 'W':
    disablePump2();
    break;
case 'z':
case 'Z':
    emergencyStopPump2();
    break;
case 'f':
case 'F':
    setPump2Direction(PUMP_DIR_FORWARD);
    break;
case 'v':
case 'V':
    setPump2Direction(PUMP_DIR_REVERSE);
    break;
case 'o':
case 'O':
    printPump2Status();
    break;

case 'u':
case 'U':
    Serial.println("\n[PUMP 3] Manual start requested.");
    if (isCase1EmergencyStopActive()) {
        Serial.println("[PUMP 3] Start rejected: Emergency Stop is
            active.");
    } else if (!isSystemOn()) {
        Serial.println("[PUMP 3] Start rejected: System switch is
            OFF.");
    } else if (isCase2ShutdownActive()) {
        Serial.println("[PUMP 3] Start rejected: System latched in
            SHUTDOWN.");
    } else if (isCase2FlushingActive()) {
        Serial.println("[PUMP 3] Start rejected: Case 2 flush
            already active.");
    } else {
        Serial.println("[PUMP 3] Start rejected: flush line valve
            has no sensor.");
        Serial.println("[PUMP 3] Use physical start button (GPIO 34)
            to start automatic sequence.");
    }
    break;

case 'i':
case 'I':
    if (isCase2FlushingActive()) {
        Serial.println("\n[PUMP 3] Stop rejected: Case 2 flush
            active (use E-Stop to halt).");
    } else if (!isSystemOn()) {
        Serial.println("\n[PUMP 3] Stop rejected: System switch is
            OFF.");
    } else if (isCase2ShutdownActive()) {
        Serial.println("\n[PUMP 3] Stop rejected: System latched in
            SHUTDOWN.");
    } else {
        disablePump3(false);
        setPump3Duty(0, false);
        Serial.println("\n[PUMP 3] Stopped by serial command.");
    }
    break;

case '\r':
case '\n':
    break;
default:
    Serial.printf("[CMD] Unknown command: '%c'. Send 'h' for
        help.\n", cmd);
    break;
}
}

static void handleMainCommandLine(const char* line) {
    if (!line || !*line) return;

    if (strcasecmp(line, "!sys_off") == 0 || strcasecmp(line, "sys_off") ==
        0 ||
        strcasecmp(line, "sys off") == 0 || strcasecmp(line, "system off")
            == 0) {
        setSystemOn(false);
        updateCase1WithInputs(isSystemOn(), isFloatSwitchTriggered());
        return;
    }

    if (strcasecmp(line, "!sys_on") == 0 || strcasecmp(line, "sys_on") == 0
        ||
        strcasecmp(line, "sys on") == 0 || strcasecmp(line, "system on") ==
            0) {
        setSystemOn(true);
        updateCase1WithInputs(isSystemOn(), isFloatSwitchTriggered());
        return;
    }

    DutyCommand cmd;
    DutyParseResult res = parseDuty5PercentCommand(line, 2, cmd);

    if (res == DUTY_PARSE_OK) {
        if (!checkPumpSafety(cmd.targetPump)) return;
        if (cmd.targetPump == 1) {
            setPump1Duty(cmd.dutyPercent);
        } else {
            setPump2Duty(cmd.dutyPercent);
        }
    } else if (res != DUTY_PARSE_NOT_A_DUTY_CMD) {
        printDutyParseError(line, cmd);
    } else {
        Serial.printf("[CMD] Unknown command: '%s'. Send 'h' for help.\n",
            line);
    }
}

static char* trimWhitespace(char* str) {
    if (!str) return nullptr;
    while (*str && isspace((unsigned char)*str)) str++;
    if (!*str) return str;
    char* end = str + strlen(str) - 1;
    while (end > str && isspace((unsigned char)*end)) end--;
    *(end + 1) = '\0';
    return str;
}

static void processIncomingLine(const char* rawLine) {
    char buf[32];
    strncpy(buf, rawLine, sizeof(buf) - 1);
    buf[sizeof(buf) - 1] = '\0';

    char* trimmed = trimWhitespace(buf);
    if (!trimmed || *trimmed == '\0') {
        return;
    }

    size_t len = strlen(trimmed);
    if (len == 1) {
        char c = trimmed[0];
        if (isCommissioningModeActive()) {
            handleCommissioningCommand(c);
        } else {
            handleSerialCommand(c);
        }
    } else {
        if (isCommissioningModeActive()) {
            handleCommissioningCommandLine(trimmed);
        } else {
            handleMainCommandLine(trimmed);
        }
    }
}

void setup() {
    Serial.begin(115200);

    // CRITICAL SAFETY INITIALIZATION:
    initPump1();
```

```
    initPump2();
    initPump3();
    initSolenoidValve();

    delay(500);

    Serial.println("\n--- LRC ESP32 Hardware Controller ---");
    Serial.println("Outputs: Active");

    initSystemSwitch();
    initFloatSwitch();
    initFlowSensor();
    initStartButton();

    initCase1Controller();

    updateCase1WithInputs(isSystemOn(), isFloatSwitchTriggered());

    printHelpMenu();
    Serial.printf("Initial State: SYSTEM -> %s | FLOAT -> %s | SOLENOID ->
        %s | CASE 1 -> %s\n\n\n",
                  isSystemOn() ? "ON" : "OFF",
                  isFloatSwitchTriggered() ? "SUMP HIGH" : "NORMAL",
                  isSolenoidOpen() ? "OPEN" : "CLOSED",
                  getCase1StateStr());
}

void loop() {
    updateSystemSwitch();
    updateFloatSwitch();
    updateFlowSensor();
    updateStartButton();

    if (wasStartButtonPressed()) {
        case1StartSequence();
    }

    if (!isCommissioningModeActive()) {
        updateCase1WithInputs(isSystemOn(), isFloatSwitchTriggered());
    }

    static char s_serialBuffer[32];
    static size_t s_serialBufferLen = 0;
    static unsigned long s_lastCharTime = 0;

    while (Serial.available() > 0) {
        char c = (char)Serial.read();

        // Universal Emergency Stop via spacebar takes effect immediately
        if (c == ' ' && s_serialBufferLen == 0) {
            if (isCommissioningModeActive()) {
                handleCommissioningCommand(' ');
            } else {
                case1TriggerEmergencyStop();
            }
            continue;
        }

        if (c == '\r' || c == '\n') {
            if (s_serialBufferLen > 0) {
                s_serialBuffer[s_serialBufferLen] = '\0';
                processIncomingLine(s_serialBuffer);
                s_serialBufferLen = 0;
            }
        } else {
            if (s_serialBufferLen < sizeof(s_serialBuffer) - 1) {
                s_serialBuffer[s_serialBufferLen++] = c;
                s_lastCharTime = millis();
            } else {
                s_serialBuffer[s_serialBufferLen] = '\0';
                processIncomingLine(s_serialBuffer);
                s_serialBufferLen = 0;
            }
        }
    }

    if (s_serialBufferLen > 0 && (millis() - s_lastCharTime >= 50)) {
        s_serialBuffer[s_serialBufferLen] = '\0';
        processIncomingLine(s_serialBuffer);
        s_serialBufferLen = 0;
    }

    if (s_csvLoggingActive) {
        if (millis() - s_lastCsvLogTime >= 2500) {
            s_lastCsvLogTime = millis();
            uint8_t pump2Pwm = isPump2Enabled() ? getPump2Duty() : 0;
            uint32_t freqHz = (uint32_t)(getFlowFrequencyHz() + 0.5f);
            Serial.printf("%lu,%u,%lu,%.2f\n",
                          (unsigned long)millis(),
                          (unsigned int)pump2Pwm,
                          (unsigned long)freqHz,
                          getFlowRateLPM());
        }
    }

    static unsigned long lastHeartbeat = 0;
    if (!s_csvLoggingActive && (millis() - lastHeartbeat >= 15000)) {
        lastHeartbeat = millis();
        if (isCommissioningModeActive()) {
            Serial.printf("[HEARTBEAT] ESP32 Alive | MODE: COMMISSIONING |
                Sys: %s | Solenoid: %s | Flow: %.2f L/min | Float: %s\n",
                          isSystemOn() ? "ON" : "OFF",
                          isSolenoidOpen() ? "OPEN" : "CLOSED",
                          getFlowRateLPM(),
                          isFloatSwitchTriggered() ? "TRIGGERED" :
                              "NORMAL");
        } else {
            Serial.printf("[HEARTBEAT] ESP32 Alive | Sys: %s | Solenoid: %s
                | Flow: %.2f L/min | Float: %s | State: %s | P1: %d%% | P2:
                %d%% | P3: %d%%\n",
                          isSystemOn() ? "ON" : "OFF",
                          isSolenoidOpen() ? "OPEN" : "CLOSED",
                          getFlowRateLPM(),
                          isFloatSwitchTriggered() ? "TRIGGERED" : "NORMAL",
                          getCase1StateStr(),
                          getCase1Pump1TargetDuty(),
                          getCase1Pump2TargetDuty(),
                          getCase1Pump3TargetDuty());
        }
    }

    delay(10);
}
```

src/case1_controller.cpp

```
#include "case1_controller.h"
#include "pump_controller.h"
#include "pump2_controller.h"
#include "pump3_controller.h"
#include "solenoid_valve.h"
#include "hardware_config.h"
#include "float_switch.h"
#include "system_switch.h"

static Case1State s_currentState = CASE1_STATE_SAFE_IDLE;
static Case1InputSource s_inputSource = CASE1_SRC_REAL_SENSOR;
static uint8_t s_pump1NormalDuty = CASE1_PUMP1_DEFAULT_NORMAL_DUTY;
static uint8_t s_pump2NormalDuty = CASE1_PUMP2_DEFAULT_NORMAL_DUTY;
static uint8_t s_pump1TargetDuty = 0;
static uint8_t s_pump2TargetDuty = 0;
static uint8_t s_pump3TargetDuty = 0;
static bool s_hardwareOutputEnabled = true;
static bool s_emergencyStopActive = false;

static bool s_lastRealTriggered = false;
static bool s_hasInitialized = false;

static unsigned long s_transitionStartTime = 0;
static bool s_transitionTimerActive = false;

static unsigned long s_flushStartTime = 0;
static uint32_t s_flushDurationMs = CASE2_FLUSH_DURATION_MS;

static void applyHardwareIfEnabled() {
    if (s_hardwareOutputEnabled) {
        if (s_pump1TargetDuty > 0) {
            setPump1Duty(s_pump1TargetDuty, false);
        } else {
            disablePump1(false);
        }

        if (s_pump2TargetDuty > 0) {
            setPump2Duty(s_pump2TargetDuty, false);
        } else {
            disablePump2(false);
        }

        if (s_pump3TargetDuty > 0) {
            setPump3Duty(s_pump3TargetDuty, false);
        } else {
            disablePump3(false);
        }
    } else {
        Serial.println("[CASE1] Hardware outputs suppressed.");
    }
}

void initCase1Controller() {
    s_currentState = CASE1_STATE_SAFE_IDLE;
    s_inputSource = CASE1_SRC_REAL_SENSOR;
    s_pump1NormalDuty = CASE1_PUMP1_DEFAULT_NORMAL_DUTY;
    s_pump2NormalDuty = CASE1_PUMP2_DEFAULT_NORMAL_DUTY;
    s_pump1TargetDuty = 0;
    s_pump2TargetDuty = 0;
    s_pump3TargetDuty = 0;
    s_hardwareOutputEnabled = true;
    s_emergencyStopActive = false;
    s_hasInitialized = false;
    s_transitionStartTime = 0;
    s_transitionTimerActive = false;
    s_flushStartTime = 0;
    s_flushDurationMs = CASE2_FLUSH_DURATION_MS;

    closeSolenoid();
    applyHardwareIfEnabled();

    Serial.println("[CASE1] Initialized in SAFE_IDLE (Pumps: OFF, Solenoid:
        CLOSED).");
}

bool case1StartSequence() {
    if (s_emergencyStopActive) {
        Serial.println("\n[START] Blocked: Emergency Stop is active.");
```

```
        return false;
    }
    if (!isSystemOn() || s_currentState == CASE1_STATE_SYSTEM_OFF) {
        Serial.println("\n[START] Blocked: System is OFF.");
        return false;
    }
    if (s_currentState == CASE1_STATE_NORMAL ||
        s_currentState == CASE1_STATE_SUMP_HIGH ||
        s_currentState == CASE1_STATE_RETURN_TO_NORMAL) {
        Serial.println("\n[START] Ignored: Automatic sequence is already
            running.");
        return false;
    }
    if (s_currentState == CASE2_STATE_FLUSHING) {
        Serial.println("\n[START] Ignored: Case 2 flush is already
            active.");
        return false;
    }
    if (s_currentState == CASE2_STATE_SHUTDOWN) {
        Serial.println("\n[START] Ignored: System latched in final shutdown.
            Reset system to re-arm.");
        return false;
    }

    Serial.println("\n[START] Physical start button detected: Starting
        automatic Case 1 -> Case 2 sequence.");
    return case1StartNormalFlow();
}

void case1TriggerEmergencyStop() {
    s_emergencyStopActive = true;
    s_currentState = CASE1_STATE_EMERGENCY_STOP;
    s_pump1TargetDuty = 0;
    s_pump2TargetDuty = 0;
    s_pump3TargetDuty = 0;

    emergencyStopPump1();
    emergencyStopPump2();
    emergencyStopPump3();
    closeSolenoid();

    Serial.println("\nE-STOP -> ACTIVE (All 3 Pumps Halted, Solenoid
        Closed)");
}

void case1ClearEmergencyStop() {
    s_emergencyStopActive = false;
    s_currentState = CASE1_STATE_SAFE_IDLE;
    s_pump1TargetDuty = 0;
    s_pump2TargetDuty = 0;
    s_pump3TargetDuty = 0;
    closeSolenoid();
    applyHardwareIfEnabled();
    Serial.println("\nE-STOP -> CLEARED (System in SAFE_IDLE, Motors OFF)");
}

bool isCase1EmergencyStopActive() {
    return s_emergencyStopActive;
}

bool triggerCase2FlushSequence() {
    if (s_emergencyStopActive) {
        Serial.println("\n[CASE 2] Cannot start flush: Emergency Stop is
            active.");
        return false;
    }

    if (s_currentState == CASE1_STATE_SYSTEM_OFF) {
        Serial.println("\n[CASE 2] Cannot start flush: System switch is
            OFF.");
        return false;
    }

    if (s_currentState == CASE2_STATE_FLUSHING) {
        Serial.println("\n[CASE 2] Flush already in progress.");
        return false;
    }

    if (s_currentState == CASE2_STATE_SHUTDOWN) {
        Serial.println("\n[CASE 2] System in shutdown. Cycle switch to
            reset.");
        return false;
    }

    if (s_currentState != CASE1_STATE_NORMAL &&
        s_currentState != CASE1_STATE_SUMP_HIGH &&
        s_currentState != CASE1_STATE_RETURN_TO_NORMAL) {
        Serial.printf("\n[CASE 2] Cannot start flush in state: %s\n",
            getCase1StateStr());
        return false;
    }

    closeSolenoid();

    s_pump1TargetDuty = s_pump1NormalDuty;
    s_pump2TargetDuty = s_pump2NormalDuty;
    s_pump3TargetDuty = CASE2_PUMP3_FLUSH_DUTY;

    s_flushStartTime = millis();
    s_currentState = CASE2_STATE_FLUSHING;
    s_transitionTimerActive = false;
    applyHardwareIfEnabled();

    Serial.println("\n[CASE 2] Automatic flush sequence started.");
    Serial.println("[CASE 2] Solenoid CLOSED (Isolating 10L process
        tank).");
    Serial.printf("[CASE 2] PUMP 3 STARTED at %d%% duty.\n",
        s_pump3TargetDuty);
    Serial.printf("[CASE 2] All three pumps running (P1: %d%%, P2: %d%%, P3:
        %d%%). Flush timer started: %lu seconds.\n",
                  s_pump1TargetDuty, s_pump2TargetDuty, s_pump3TargetDuty,
                      (unsigned long)(s_flushDurationMs / 1000UL));

    return true;
}

bool isCase2FlushingActive() {
    return (s_currentState == CASE2_STATE_FLUSHING);
}

bool isCase2ShutdownActive() {
    return (s_currentState == CASE2_STATE_SHUTDOWN);
}

unsigned long getCase2FlushRemainingMs() {
    if (s_currentState == CASE2_STATE_FLUSHING) {
        unsigned long elapsed = millis() - s_flushStartTime;
        if (elapsed >= s_flushDurationMs) {
            return 0;
        }
        return (s_flushDurationMs - elapsed);
    }
    return 0;
}

uint32_t getCase2FlushDurationMs() {
    return s_flushDurationMs;
}

void setCase2FlushDurationMs(uint32_t ms) {
    s_flushDurationMs = ms;
    Serial.printf("[CASE 2] Flush duration updated to %lu ms\n", (unsigned
        long)s_flushDurationMs);
}

bool isCase1TransitionDelayActive() {
    return (s_currentState == CASE1_STATE_RETURN_TO_NORMAL &&
        s_transitionTimerActive);
}

unsigned long getCase1TransitionRemainingMs() {
    if (s_currentState == CASE1_STATE_RETURN_TO_NORMAL &&
        s_transitionTimerActive) {
        unsigned long elapsed = millis() - s_transitionStartTime;
        if (elapsed >= CASE1_RETURN_DELAY_MS) {
            return 0;
        }
        return (CASE1_RETURN_DELAY_MS - elapsed);
    }
    return 0;
}

void updateCase1WithInputs(bool isSystemSwitchOn, bool
    isFloatSwitchTriggered) {
    // 1. HIGHEST PRIORITY: Emergency Stop
    if (s_emergencyStopActive) {
        s_pump1TargetDuty = 0;
        s_pump2TargetDuty = 0;
        s_pump3TargetDuty = 0;
        s_transitionTimerActive = false;
        return;
    }

    if (!isSystemSwitchOn) {
        if (s_currentState != CASE1_STATE_SYSTEM_OFF) {
            s_currentState = CASE1_STATE_SYSTEM_OFF;
            s_pump1TargetDuty = 0;
            s_pump2TargetDuty = 0;
            s_pump3TargetDuty = 0;
            s_transitionTimerActive = false;
            closeSolenoid();

            Serial.println("\nSYSTEM -> OFF");
            Serial.println("CASE 1 -> SYSTEM OFF");
            Serial.println("P1: 0% | P2: 0% | P3: 0% | Solenoid: CLOSED");
            applyHardwareIfEnabled();
        }
        return;
    }

    if (s_currentState == CASE2_STATE_FLUSHING) {
        unsigned long elapsed = millis() - s_flushStartTime;
        if (elapsed >= s_flushDurationMs) {
            s_currentState = CASE2_STATE_SHUTDOWN;
            s_pump1TargetDuty = 0;
            s_pump2TargetDuty = 0;
            s_pump3TargetDuty = 0;
            closeSolenoid();
            disablePump1(false);
            disablePump2(false);
            disablePump3(false);
            applyHardwareIfEnabled();
```

```
        Serial.printf("\n[CASE 2] %lu-second flush complete.\n",
            (unsigned long)(s_flushDurationMs / 1000UL));
        Serial.println("[CASE 2] All pumps STOPPED. Motor drivers
            disabled.");
        Serial.println("[CASE 2] Solenoid remains CLOSED. System latched
            in SHUTDOWN.");
    }
    return;
}

if (s_currentState == CASE2_STATE_SHUTDOWN) {
    return;
}

// CRITICAL SAFETY FIX: Turning switch ON or booting must NEVER
    automatically start motors!
if (s_currentState == CASE1_STATE_SYSTEM_OFF || !s_hasInitialized) {
    s_hasInitialized = true;
    s_lastRealTriggered = isFloatSwitchTriggered;
    s_inputSource = CASE1_SRC_REAL_SENSOR;
    s_currentState = CASE1_STATE_SAFE_IDLE;
    s_pump1TargetDuty = 0;
    s_pump2TargetDuty = 0;
    s_pump3TargetDuty = 0;
    s_transitionTimerActive = false;
    closeSolenoid();
    applyHardwareIfEnabled();

    Serial.println("\nSYSTEM -> ON");
    Serial.println("CASE 1 -> SAFE IDLE");
    Serial.println("P1: 0% | P2: 0% | P3: 0% | Solenoid: CLOSED");
    Serial.println("[SAFETY] Motors idle. Press physical start button
        (GPIO 34) to start sequence.");
    return;
}

if (s_currentState == CASE1_STATE_SAFE_IDLE) {
    s_lastRealTriggered = isFloatSwitchTriggered;
    return;
}

if (s_currentState == CASE1_STATE_RETURN_TO_NORMAL) {
    if (isFloatSwitchTriggered) {
        s_transitionTimerActive = false;
        s_lastRealTriggered = true;
        s_currentState = CASE1_STATE_SUMP_HIGH;
        s_pump1TargetDuty = 0;
        s_pump2TargetDuty = s_pump2NormalDuty;
        s_pump3TargetDuty = 0;
        openSolenoid();
        applyHardwareIfEnabled();

        Serial.println("\nFLOAT -> SUMP HIGH (Return delay canceled:
            fluid high again)");
        Serial.println("CASE 1 -> SUMP HIGH");
        Serial.printf("P1: 0% | P2: %d%% | Solenoid: OPEN\n",
            s_pump2TargetDuty);
        return;
    }

    if (s_transitionTimerActive) {
        unsigned long elapsed = millis() - s_transitionStartTime;
        if (elapsed >= CASE1_RETURN_DELAY_MS) {
            s_transitionTimerActive = false;
            Serial.println("\n[AUTO-SEQUENCE] 10-second return delay
                completed.");
            Serial.println("[AUTO-SEQUENCE] Automatically transitioning
                to Case 2 flush sequence.");
            triggerCase2FlushSequence();
            return;
        }
    }
    return;
}

if (s_currentState == CASE1_STATE_NORMAL || s_currentState ==
    CASE1_STATE_SUMP_HIGH) {
    if (s_inputSource == CASE1_SRC_REAL_SENSOR &&
        (isFloatSwitchTriggered != s_lastRealTriggered)) {
        s_lastRealTriggered = isFloatSwitchTriggered;
        s_pump3TargetDuty = 0;

        if (isFloatSwitchTriggered) {
            s_transitionTimerActive = false;
            s_currentState = CASE1_STATE_SUMP_HIGH;
            s_pump1TargetDuty = 0;
            s_pump2TargetDuty = s_pump2NormalDuty;
            openSolenoid();
            applyHardwareIfEnabled();
            Serial.println("\nFLOAT -> SUMP HIGH");
            Serial.println("CASE 1 -> SUMP HIGH");
            Serial.printf("P1: 0% | P2: %d%% | Solenoid: OPEN\n",
                s_pump2TargetDuty);
        } else {
            s_currentState = CASE1_STATE_RETURN_TO_NORMAL;
            s_pump1TargetDuty = s_pump1NormalDuty;
            s_pump2TargetDuty = s_pump2NormalDuty;
            openSolenoid();
            s_transitionStartTime = millis();
            s_transitionTimerActive = true;
            applyHardwareIfEnabled();
            Serial.println("\nFLOAT -> NORMAL");
            Serial.println("CASE 1 -> RETURN TO NORMAL");
            Serial.println("[AUTO-SEQUENCE] 10-second delay started.
                Solenoid OPEN. Case 2 starts automatically.");
            Serial.printf("P1: %d%% | P2: %d%% | Solenoid: OPEN\n",
                s_pump1TargetDuty, s_pump2TargetDuty);
        }
    }
}

bool case1StartNormalFlow() {
    if (s_emergencyStopActive) {
        Serial.println("[SAFETY] Blocked: Cannot start Normal Flow while
            Emergency Stop is active.");
        return false;
    }
    if (!isSystemOn() || s_currentState == CASE1_STATE_SYSTEM_OFF) {
        Serial.println("[SAFETY] Blocked: Cannot start Normal Flow while
            System Switch is OFF.");
        return false;
    }
    if (s_currentState == CASE2_STATE_FLUSHING) {
        Serial.println("[SAFETY] Blocked: Cannot start Normal Flow while
            Case 2 Flush is running.");
        return false;
    }
    if (s_currentState == CASE2_STATE_SHUTDOWN) {
        Serial.println("[SAFETY] Blocked: System in shutdown. Cycle switch
            to reset.");
        return false;
    }

    s_inputSource = CASE1_SRC_REAL_SENSOR;
    s_pump3TargetDuty = 0;
    bool floatTrig = isFloatSwitchTriggered();
    s_lastRealTriggered = floatTrig;

    if (!floatTrig) {
        s_currentState = CASE1_STATE_NORMAL;
        s_pump1TargetDuty = s_pump1NormalDuty;
        s_pump2TargetDuty = s_pump2NormalDuty;
        openSolenoid();
        Serial.println("\nCASE 1 -> NORMAL");
        Serial.printf("NORMAL FLOW -> Pump 1: %d%% | Pump 2: %d%% |
            Solenoid: OPEN\n", s_pump1TargetDuty, s_pump2TargetDuty);
    } else {
        s_currentState = CASE1_STATE_SUMP_HIGH;
        s_pump1TargetDuty = 0;
        s_pump2TargetDuty = s_pump2NormalDuty;
        openSolenoid();
        Serial.println("\nCASE 1 -> SUMP HIGH");
        Serial.printf("NORMAL FLOW -> Pump 1: 0% | Pump 2: %d%% | Solenoid:
            OPEN (Sump High)\n", s_pump2TargetDuty);
    }

    applyHardwareIfEnabled();
    return true;
}

static bool canRunSimulation() {
    if (s_emergencyStopActive) {
        Serial.println("[SAFETY] Blocked: Cannot simulate Case 1 while
            Emergency Stop is active.");
        return false;
    }
    if (s_currentState == CASE1_STATE_SYSTEM_OFF) {
        Serial.println("[SAFETY] Blocked: Cannot simulate Case 1 while
            System Switch is OFF.");
        return false;
    }
    if (s_currentState == CASE2_STATE_FLUSHING) {
        Serial.println("[SAFETY] Blocked: Cannot simulate Case 1 while Case
            2 Flush is running.");
        return false;
    }
    if (s_currentState == CASE2_STATE_SHUTDOWN) {
        Serial.println("[SAFETY] Blocked: Cannot simulate Case 1 while in
            Case 2 Shutdown.");
        return false;
    }
    return true;
}

void case1TriggerSimulatedNormal() {
    if (!canRunSimulation()) return;

    s_inputSource = CASE1_SRC_SIMULATION;
    s_currentState = CASE1_STATE_NORMAL;
    s_pump1TargetDuty = s_pump1NormalDuty;
    s_pump2TargetDuty = s_pump2NormalDuty;
    s_pump3TargetDuty = 0;
    openSolenoid();

    Serial.println("\nCASE 1 -> NORMAL");
    Serial.printf("P1: %d%% | P2: %d%% | P3: 0%% | Solenoid: OPEN\n",
        s_pump1TargetDuty, s_pump2TargetDuty);
    applyHardwareIfEnabled();
}

void case1TriggerSimulatedSumpHigh() {
    if (!canRunSimulation()) return;
```

```
    s_inputSource = CASE1_SRC_SIMULATION;
    s_transitionTimerActive = false;
    s_currentState = CASE1_STATE_SUMP_HIGH;
    s_pump1TargetDuty = 0;
    s_pump2TargetDuty = s_pump2NormalDuty;
    s_pump3TargetDuty = 0;
    openSolenoid();

    Serial.println("\nCASE 1 -> SUMP HIGH (Simulated)");
    Serial.printf("P1: 0% | P2: %d%% | P3: 0%% | Solenoid: OPEN\n",
        s_pump2TargetDuty);
    applyHardwareIfEnabled();
}

void case1TriggerSimulatedReturnToNormal() {
    if (!canRunSimulation()) return;

    s_inputSource = CASE1_SRC_SIMULATION;
    s_currentState = CASE1_STATE_RETURN_TO_NORMAL;
    s_pump1TargetDuty = s_pump1NormalDuty;
    s_pump2TargetDuty = s_pump2NormalDuty;
    s_pump3TargetDuty = 0;
    openSolenoid();
    s_transitionStartTime = millis();
    s_transitionTimerActive = true;

    Serial.println("\nCASE 1 -> RETURN TO NORMAL (Simulated)");
    Serial.println("[AUTO-SEQUENCE] 10-second delay started. Solenoid OPEN.
        Flush starts automatically.");
    Serial.printf("P1: %d%% | P2: %d%% | P3: 0%% | Solenoid: OPEN\n",
        s_pump1TargetDuty, s_pump2TargetDuty);
    applyHardwareIfEnabled();
}

Case1State getCase1State() {
    return s_currentState;
}

const char* getCase1StateStr() {
    switch (s_currentState) {
        case CASE1_STATE_SAFE_IDLE:        return "SAFE_IDLE";
        case CASE1_STATE_EMERGENCY_STOP:   return "EMERGENCY_STOP";
        case CASE1_STATE_SYSTEM_OFF:       return "SYSTEM_OFF";
        case CASE1_STATE_NORMAL:           return "NORMAL";
        case CASE1_STATE_SUMP_HIGH:        return "SUMP_HIGH";
        case CASE1_STATE_RETURN_TO_NORMAL: return "RETURN_TO_NORMAL";
        case CASE2_STATE_FLUSHING:         return "CASE2_FLUSHING";
        case CASE2_STATE_SHUTDOWN:         return "CASE2_SHUTDOWN";
        default:                           return "UNKNOWN";
    }
}

Case1InputSource getCase1InputSource() {
    return s_inputSource;
}

const char* getCase1InputSourceStr() {
    return (s_inputSource == CASE1_SRC_REAL_SENSOR) ? "REAL_SENSOR" :
        "SIMULATION";
}

uint8_t getCase1Pump1TargetDuty() {
    return s_pump1TargetDuty;
}

uint8_t getCase1Pump2TargetDuty() {
    return s_pump2TargetDuty;
}

uint8_t getCase1Pump3TargetDuty() {
    return s_pump3TargetDuty;
}

void setCase1Pump1NormalDuty(uint8_t duty) {
    if (duty > 100) duty = 100;
    s_pump1NormalDuty = duty;
    Serial.printf("[CASE1] Pump 1 normal duty set to %d%%\n",
        s_pump1NormalDuty);
}

void setCase1Pump2NormalDuty(uint8_t duty) {
    if (duty > 100) duty = 100;
    s_pump2NormalDuty = duty;
    Serial.printf("[CASE1] Pump 2 normal duty set to %d%%\n",
        s_pump2NormalDuty);
}

void setCase1HardwareOutputEnabled(bool enabled) {
    s_hardwareOutputEnabled = enabled;
    setSolenoidHardwareOutputEnabled(enabled);
    applyHardwareIfEnabled();
    Serial.printf("[CASE1] Hardware output: %s\n", enabled ? "ENABLED" :
        "DISABLED");
}

bool isCase1HardwareOutputEnabled() {
    return s_hardwareOutputEnabled;
}

void printCase1Status() {
    Serial.println("--- System Status ---");
    Serial.printf("  State:    %s\n", getCase1StateStr());
    Serial.printf("  E-Stop:   %s\n", s_emergencyStopActive ? "ACTIVE" :
        "CLEARED");
    Serial.printf("  Pump 1:   %d%%\n", s_pump1TargetDuty);
    Serial.printf("  Pump 2:   %d%%\n", s_pump2TargetDuty);
    Serial.printf("  Pump 3:   %d%%\n", s_pump3TargetDuty);
    Serial.printf("  Solenoid: %s\n", isSolenoidOpen() ? "OPEN" : "CLOSED");
    if (s_currentState == CASE1_STATE_RETURN_TO_NORMAL &&
        s_transitionTimerActive) {
        Serial.printf("  Delay:    %lu ms until automatic Case 2 flush\n",
            getCase1TransitionRemainingMs());
    }
    if (s_currentState == CASE2_STATE_FLUSHING) {
        Serial.printf("  Flush:    %lu ms remaining\n",
            getCase2FlushRemainingMs());
    }
    Serial.printf("  Hardware: %s\n", s_hardwareOutputEnabled ? "ENABLED" :
        "DISABLED");
}
```

src/float_switch.cpp

```
#include "float_switch.h"

static int s_debouncedReading = HIGH;
static int s_lastRawReading = HIGH;
static unsigned long s_lastDebounceTime = 0;

void initFloatSwitch() {
    // GPIO 35 has an external 10k pull-up resistor on the PCB
    pinMode(FLOAT_SWITCH_PIN, INPUT);

    int raw = digitalRead(FLOAT_SWITCH_PIN);
    s_debouncedReading = raw;
    s_lastRawReading = raw;
    s_lastDebounceTime = millis();

    Serial.println("[FLOAT_SWITCH] Initialized (GPIO 35).");
    Serial.printf("[FLOAT_SWITCH] State: %s\n", isFloatSwitchTriggered() ?
        "SUMP HIGH" : "NORMAL");
}

void updateFloatSwitch() {
    int currentRaw = digitalRead(FLOAT_SWITCH_PIN);

    if (currentRaw != s_lastRawReading) {
        s_lastDebounceTime = millis();
        s_lastRawReading = currentRaw;
    }

    if ((millis() - s_lastDebounceTime) >= FLOAT_SWITCH_DEBOUNCE_MS) {
        s_debouncedReading = currentRaw;
    }
}

int readFloatSwitchRaw() {
    return digitalRead(FLOAT_SWITCH_PIN);
}

bool isFloatSwitchTriggered() {
    return (s_debouncedReading == FLOAT_SWITCH_TRIGGERED_LEVEL);
}

bool isFloatSwitchNormal() {
    return !isFloatSwitchTriggered();
}

void printFloatSwitchStatus() {
    int raw = digitalRead(FLOAT_SWITCH_PIN);
    bool triggered = isFloatSwitchTriggered();

    Serial.println("--- Float Switch (GPIO 35) ---");
    Serial.printf("  Raw:   %s (%d)\n", (raw == HIGH) ? "HIGH" : "LOW",
        raw);
    Serial.printf("  State: %s\n", triggered ? "TRIGGERED (SUMP HIGH)" :
        "NORMAL");
}
```

src/start_button.cpp

```
#include "start_button.h"

static int s_debouncedReading = HIGH;
static int s_lastRawReading = HIGH;
static unsigned long s_lastDebounceTime = 0;
static bool s_pressDetected = false;

void initStartButton() {
    if (IS_PIN_VALID(START_BUTTON_PIN)) {
        // ESP32 GPIO 34 is input-only (GPI) and does NOT have internal
            pull-ups in silicon.
        // MUST configure as INPUT (never INPUT_PULLUP). The PCB schematic
            provides an
        // external 10 kΩ pull-up to 3.3V (R10) and 100 nF RC filter
            capacitor (C9) to GND.
        pinMode(START_BUTTON_PIN, INPUT);

        int raw = digitalRead(START_BUTTON_PIN);
        s_debouncedReading = raw;
        s_lastRawReading = raw;
        s_lastDebounceTime = millis();
        s_pressDetected = false;
```

```
        Serial.println("[START_BUTTON] Initialized (GPIO 34, external 10k
            pull-up). Starts full auto sequence.");
    }
}

void updateStartButton() {
    if (!IS_PIN_VALID(START_BUTTON_PIN)) return;

    int currentRaw = digitalRead(START_BUTTON_PIN);

    if (currentRaw != s_lastRawReading) {
        s_lastDebounceTime = millis();
        s_lastRawReading = currentRaw;
    }

    if ((millis() - s_lastDebounceTime) >= START_BUTTON_DEBOUNCE_MS) {
        if (currentRaw != s_debouncedReading) {
            if (currentRaw == START_BUTTON_ACTIVE_LEVEL &&
                s_debouncedReading != START_BUTTON_ACTIVE_LEVEL) {
                s_pressDetected = true;
            }
            s_debouncedReading = currentRaw;
        }
    }
}

bool wasStartButtonPressed() {
    if (s_pressDetected) {
        s_pressDetected = false;
        return true;
    }
    return false;
}

bool isStartButtonPhysicallyPressed() {
    return (s_debouncedReading == START_BUTTON_ACTIVE_LEVEL);
}

int getStartButtonRawState() {
    if (!IS_PIN_VALID(START_BUTTON_PIN)) return HIGH;
    return digitalRead(START_BUTTON_PIN);
}

void printStartButtonStatus() {
    int raw = getStartButtonRawState();
    bool pressed = isStartButtonPhysicallyPressed();

    Serial.println("--- Start Button (GPIO 34) ---");
    Serial.printf("  Raw:   %s (%d)\n", (raw == HIGH) ? "HIGH" : "LOW",
        raw);
    Serial.printf("  State: %s\n", pressed ? "PRESSED" : "RELEASED");
    Serial.println("  Role:  Starts complete Case 1 -> Case 2 automatic
        sequence");
}
```

src/system_switch.cpp

```
#include "system_switch.h"

static bool s_systemOn = true;

void initSystemSwitch() {
    s_systemOn = true;
    Serial.println("[SYSTEM] Initialized (Software control).");
    Serial.printf("[SYSTEM] State: %s\n", s_systemOn ? "SYSTEM ON" : "SYSTEM
        OFF");
}

void updateSystemSwitch() {
}

bool isSystemOn() {
    return s_systemOn;
}

void setSystemOn(bool enabled) {
    s_systemOn = enabled;
}

void toggleSystemOn() {
    s_systemOn = !s_systemOn;
}

int getSystemSwitchRawState() {
    return s_systemOn ? HIGH : LOW;
}

void printSystemSwitchStatus() {
    Serial.println("--- System Enable (Software Control) ---");
    Serial.printf("  State: %s\n", s_systemOn ? "SYSTEM ON" : "SYSTEM OFF");
}
```

src/pump_controller.cpp

```
#include "pump_controller.h"
#include "bts7960_pump.h"
#include "case1_controller.h"

uint32_t Bts7960Pump::calculateRawDuty(uint8_t percent) const {
    if (percent > 100) percent = 100;
    const uint32_t maxCount = (1UL << resBits) - 1;
    return (percent * maxCount) / 100;
}

void Bts7960Pump::init() {
    enabled = false;
    duty = 0;
    direction = PUMP_DIR_FORWARD;

    if (IS_PIN_VALID(lEnPin)) {
        digitalWrite(lEnPin, LOW);
        pinMode(lEnPin, OUTPUT);
    }
    if (IS_PIN_VALID(rEnPin)) {
        digitalWrite(rEnPin, LOW);
        pinMode(rEnPin, OUTPUT);
    }

    if (IS_PIN_VALID(lPwmPin)) {
        ledcSetup(lChannel, pwmFreq, resBits);
        ledcAttachPin(lPwmPin, lChannel);
        ledcWrite(lChannel, 0);
    }
    if (IS_PIN_VALID(rPwmPin)) {
        ledcSetup(rChannel, pwmFreq, resBits);
        ledcAttachPin(rPwmPin, rChannel);
        ledcWrite(rChannel, 0);
    }

    Serial.println(initTag);
}

void Bts7960Pump::applyOutputs() {
    if (!isCase1HardwareOutputEnabled() || !enabled) {
        if (IS_PIN_VALID(lPwmPin)) ledcWrite(lChannel, 0);
        if (IS_PIN_VALID(rPwmPin)) ledcWrite(rChannel, 0);
        if (IS_PIN_VALID(lEnPin))  digitalWrite(lEnPin, LOW);
        if (IS_PIN_VALID(rEnPin))  digitalWrite(rEnPin, LOW);
        return;
    }

    if (IS_PIN_VALID(lEnPin)) digitalWrite(lEnPin, HIGH);
    if (IS_PIN_VALID(rEnPin)) digitalWrite(rEnPin, HIGH);

    if (duty == 0) {
        if (IS_PIN_VALID(lPwmPin)) ledcWrite(lChannel, 0);
        if (IS_PIN_VALID(rPwmPin)) ledcWrite(rChannel, 0);
    } else if (direction == PUMP_DIR_FORWARD) {
        if (IS_PIN_VALID(lPwmPin)) ledcWrite(lChannel,
            calculateRawDuty(duty));
        if (IS_PIN_VALID(rPwmPin)) ledcWrite(rChannel, 0);
    } else {
        if (IS_PIN_VALID(lPwmPin)) ledcWrite(lChannel, 0);
        if (IS_PIN_VALID(rPwmPin)) ledcWrite(rChannel,
            calculateRawDuty(duty));
    }
}

void Bts7960Pump::enable(bool printFeedback) {
    enabled = true;
    applyOutputs();
    if (printFeedback) Serial.printf("%s -> ENABLED\n", name);
}

void Bts7960Pump::disable(bool printFeedback) {
    enabled = false;
    applyOutputs();
    if (printFeedback) Serial.printf("%s -> DISABLED\n", name);
}

void Bts7960Pump::emergencyStop() {
    duty = 0;
    enabled = false;
    applyOutputs();
    Serial.printf("%s -> STOP\n", name);
}

bool Bts7960Pump::setDuty(uint8_t percent, bool printFeedback) {
    if (percent > 100) percent = 100;
    duty = percent;
    if (percent > 0 && !enabled) enabled = true;
    applyOutputs();

    if (printFeedback) {
        if (duty == 0) Serial.printf("%s -> STOP\n", name);
        else Serial.printf("%s -> %d%%\n", name, duty);
    }
    return true;
}

void Bts7960Pump::setDirection(PumpDirection dir) {
    direction = dir;
    applyOutputs();
    Serial.printf("%s Direction -> %s\n", name, (dir == PUMP_DIR_FORWARD) ?
        "FORWARD" : "REVERSE");
}

void Bts7960Pump::printStatus() const {
    Serial.println(statusTag);
    Serial.printf("  State:     %s\n", enabled ? "ENABLED" : "DISABLED");
    Serial.printf("  Direction: %s\n", (direction == PUMP_DIR_FORWARD) ?
        "FORWARD" : "REVERSE");
    Serial.printf("  Duty:      %d%% (PWM: %lu/255)\n", duty, (unsigned
        long)calculateRawDuty(duty));
```

```
    Serial.printf("  Hardware:  %s\n", isCase1HardwareOutputEnabled() ?
        "ENABLED" : "DISABLED");
}

Bts7960Pump g_pump1 = {
    "Pump 1",
    "[PUMP 1] Initialized (Driver U2, stopped).",
    "--- Pump 1 (U2) ---",
    PUMP1_L_PWM_PIN, PUMP1_R_PWM_PIN,
    PUMP1_L_EN_PIN, PUMP1_R_EN_PIN,
    PUMP1_L_PWM_CHANNEL, PUMP1_R_PWM_CHANNEL,
    PUMP1_PWM_FREQ_HZ, PUMP1_PWM_RESOLUTION_BITS,
    PUMP1_DEFAULT_DIRECTION, false, 0
};

Bts7960Pump g_pump2 = {
    "Pump 2",
    "[PUMP 2] Initialized (Driver U3, stopped).",
    "--- Pump 2 (U3) ---",
    PUMP2_L_PWM_PIN, PUMP2_R_PWM_PIN,
    PUMP2_L_EN_PIN, PUMP2_R_EN_PIN,
    PUMP2_L_PWM_CHANNEL, PUMP2_R_PWM_CHANNEL,
    PUMP2_PWM_FREQ_HZ, PUMP2_PWM_RESOLUTION_BITS,
    PUMP2_DEFAULT_DIRECTION, false, 0
};

Bts7960Pump g_pump3 = {
    "Pump 3",
    "[PUMP 3] Initialized (Driver U4, stopped).",
    "--- Pump 3 (U4) ---",
    PUMP3_L_PWM_PIN, PUMP3_R_PWM_PIN,
    PUMP3_L_EN_PIN, PUMP3_R_EN_PIN,
    PUMP3_L_PWM_CHANNEL, PUMP3_R_PWM_CHANNEL,
    PUMP3_PWM_FREQ_HZ, PUMP3_PWM_RESOLUTION_BITS,
    PUMP3_DEFAULT_DIRECTION, false, 0
};

void initPump1() { g_pump1.init(); }
void enablePump1(bool printFeedback) { g_pump1.enable(printFeedback); }
void disablePump1(bool printFeedback) { g_pump1.disable(printFeedback); }
void emergencyStopPump1() { g_pump1.emergencyStop(); }
bool setPump1Duty(uint8_t percent, bool printFeedback) { return
    g_pump1.setDuty(percent, printFeedback); }
void setPump1Direction(PumpDirection dir) { g_pump1.setDirection(dir); }
PumpDirection getPump1Direction() { return g_pump1.direction; }
const char* getPump1DirectionStr() {
    return (g_pump1.direction == PUMP_DIR_FORWARD) ? "FORWARD (L_PWM
        Active)" : "REVERSE (R_PWM Active)";
}
bool isPump1Enabled() { return g_pump1.enabled; }
uint8_t getPump1Duty() { return g_pump1.duty; }
void printPump1Status() { g_pump1.printStatus(); }
```

src/pump2_controller.cpp

```
#include "pump2_controller.h"
#include "bts7960_pump.h"

void initPump2() { g_pump2.init(); }
void enablePump2(bool printFeedback) { g_pump2.enable(printFeedback); }
void disablePump2(bool printFeedback) { g_pump2.disable(printFeedback); }
void emergencyStopPump2() { g_pump2.emergencyStop(); }
bool setPump2Duty(uint8_t percent, bool printFeedback) { return
    g_pump2.setDuty(percent, printFeedback); }
void setPump2Direction(PumpDirection dir) { g_pump2.setDirection(dir); }
PumpDirection getPump2Direction() { return g_pump2.direction; }
const char* getPump2DirectionStr() {
    return (g_pump2.direction == PUMP_DIR_FORWARD) ? "FORWARD (L_PWM
        Active)" : "REVERSE (R_PWM Active)";
}
bool isPump2Enabled() { return g_pump2.enabled; }
uint8_t getPump2Duty() { return g_pump2.duty; }
void printPump2Status() { g_pump2.printStatus(); }
```

src/pump3_controller.cpp

```
#include "pump3_controller.h"
#include "bts7960_pump.h"

void initPump3() { g_pump3.init(); }
void enablePump3(bool printFeedback) { g_pump3.enable(printFeedback); }
void disablePump3(bool printFeedback) { g_pump3.disable(printFeedback); }
void emergencyStopPump3() { g_pump3.emergencyStop(); }
bool setPump3Duty(uint8_t percent, bool printFeedback) { return
    g_pump3.setDuty(percent, printFeedback); }
void setPump3Direction(PumpDirection dir) { g_pump3.setDirection(dir); }
PumpDirection getPump3Direction() { return g_pump3.direction; }
const char* getPump3DirectionStr() {
    return (g_pump3.direction == PUMP_DIR_FORWARD) ? "FORWARD (L_PWM
        Active)" : "REVERSE (R_PWM Active)";
}
bool isPump3Enabled() { return g_pump3.enabled; }
uint8_t getPump3Duty() { return g_pump3.duty; }
void printPump3Status() { g_pump3.printStatus(); }
```

src/solenoid_valve.cpp

```
#include "solenoid_valve.h"
#include "case1_controller.h"
#include "system_switch.h"

static bool s_solenoidOpen = false;
static bool s_hardwareOutputEnabled = true;

static uint8_t getOpenLevel() {
    return SOLENOID_ACTIVE_HIGH ? HIGH : LOW;
}

static uint8_t getClosedLevel() {
    return SOLENOID_ACTIVE_HIGH ? LOW : HIGH;
}

static void applyHardwareOutputs() {
    if (s_hardwareOutputEnabled) {
        if (IS_PIN_VALID(SOLENOID_VALVE_PIN)) {
            digitalWrite(SOLENOID_VALVE_PIN, s_solenoidOpen ? getOpenLevel()
                : getClosedLevel());
        }
    } else {
        if (IS_PIN_VALID(SOLENOID_VALVE_PIN)) {
            digitalWrite(SOLENOID_VALVE_PIN, getClosedLevel());
        }
        Serial.println("[SOLENOID] Dry-run: output suppressed.");
    }
}

void initSolenoidValve() {
    s_solenoidOpen = false;
    s_hardwareOutputEnabled = true;

    if (IS_PIN_VALID(SOLENOID_VALVE_PIN)) {
        digitalWrite(SOLENOID_VALVE_PIN, getClosedLevel());
        pinMode(SOLENOID_VALVE_PIN, OUTPUT);
        digitalWrite(SOLENOID_VALVE_PIN, getClosedLevel());
    }

    Serial.println("[SOLENOID] Initialized (GPIO 27, CLOSED).");
}

void openSolenoid() {
    if (isCase1EmergencyStopActive()) {
        Serial.println("[SAFETY] Blocked: Cannot open solenoid while
            Emergency Stop is active.");
        return;
    }
    if (!isSystemOn()) {
        Serial.println("[SAFETY] Blocked: Cannot open solenoid while System
            Switch is OFF.");
        return;
    }

    s_solenoidOpen = true;
    applyHardwareOutputs();
    Serial.println("SOLENOID -> OPEN");
}

void closeSolenoid() {
    s_solenoidOpen = false;
    applyHardwareOutputs();
    Serial.println("SOLENOID -> CLOSED");
}

void toggleSolenoid() {
    if (s_solenoidOpen) {
        closeSolenoid();
    } else {
        openSolenoid();
    }
}

bool isSolenoidOpen() {
    return s_solenoidOpen;
}

void setSolenoidHardwareOutputEnabled(bool enabled) {
    s_hardwareOutputEnabled = enabled;
    Serial.printf("[SOLENOID] Hardware output: %s\n", enabled ? "ENABLED" :
        "DISABLED");
    applyHardwareOutputs();
}

bool isSolenoidHardwareOutputEnabled() {
    return s_hardwareOutputEnabled;
}

void printSolenoidStatus() {
    Serial.println("--- Solenoid Valve (GPIO 27) ---");
    Serial.printf("  State:    %s\n", s_solenoidOpen ? "OPEN" : "CLOSED");
    Serial.printf("  Hardware: %s\n", s_hardwareOutputEnabled ? "ENABLED" :
        "DISABLED");
}
```

src/flow_sensor.cpp

```
#include "flow_sensor.h"

static volatile uint32_t s_isrWindowPulses = 0;
static volatile uint32_t s_isrTotalPulses = 0;

static float s_liveFrequencyHz = 0.0f;
static float s_liveFlowLPM = 0.0f;
static unsigned long s_lastWindowTime = 0;

void IRAM_ATTR flowSensorISR() {
```

```
    s_isrWindowPulses++;
    s_isrTotalPulses++;
}

void initFlowSensor() {
    pinMode(FLOW_SENSOR_PIN, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(FLOW_SENSOR_PIN), flowSensorISR,
        RISING);

    s_liveFrequencyHz = 0.0f;
    s_liveFlowLPM = 0.0f;
    s_lastWindowTime = millis();

    Serial.println("[FLOW_SENSOR] Initialized (GPIO 14, YF-S401).");
}

void updateFlowSensor() {
    unsigned long now = millis();
    unsigned long elapsedMs = now - s_lastWindowTime;

    if (elapsedMs >= FLOW_SENSOR_UPDATE_INTERVAL_MS) {
        noInterrupts();
        uint32_t pulses = s_isrWindowPulses;
        s_isrWindowPulses = 0;
        interrupts();

        s_lastWindowTime = now;

        if (pulses == 0) {
            s_liveFrequencyHz = 0.0f;
            s_liveFlowLPM = 0.0f;
        } else {
            s_liveFrequencyHz = ((float)pulses * 1000.0f) /
                (float)elapsedMs;
            s_liveFlowLPM = s_liveFrequencyHz / FLOW_SENSOR_HZ_PER_LPM;
        }
    }
}

float getFlowRateLPM() {
    return s_liveFlowLPM;
}

float getFlowFrequencyHz() {
    return s_liveFrequencyHz;
}

uint32_t getTotalPulseCount() {
    noInterrupts();
    uint32_t total = s_isrTotalPulses;
    interrupts();
    return total;
}

void resetFlowPulseCount() {
    noInterrupts();
    s_isrTotalPulses = 0;
    s_isrWindowPulses = 0;
    interrupts();
    s_liveFrequencyHz = 0.0f;
    s_liveFlowLPM = 0.0f;
    Serial.println("[FLOW_SENSOR] Cumulative pulse counter reset to 0.");
}

FlowRangeStatus getFlowRangeStatus() {
    if (s_liveFlowLPM <= 0.001f) {
        return FLOW_RANGE_ZERO;
    } else if (s_liveFlowLPM < FLOW_SENSOR_MIN_RATED_LPM) {
        return FLOW_RANGE_LOW;
    } else if (s_liveFlowLPM > FLOW_SENSOR_MAX_RATED_LPM) {
        return FLOW_RANGE_HIGH;
    }
    return FLOW_RANGE_NORMAL;
}

const char* getFlowRangeStatusStr() {
    switch (getFlowRangeStatus()) {
        case FLOW_RANGE_ZERO:   return "ZERO (No Flow)";
        case FLOW_RANGE_LOW:    return "BELOW RATED RANGE (<0.3 L/min)";
        case FLOW_RANGE_NORMAL: return "IN RANGE (0.3 - 6.0 L/min)";
        case FLOW_RANGE_HIGH:   return "ABOVE RATED RANGE (>6.0 L/min)";
        default:                return "UNKNOWN";
    }
}

void printFlowSensorStatus() {
    uint32_t totalPulses = getTotalPulseCount();

    Serial.println("--- Flow Sensor (GPIO 14) ---");
    Serial.printf("  Pulses:    %lu\n", (unsigned long)totalPulses);
    Serial.printf("  Frequency: %.2f Hz\n", s_liveFrequencyHz);
    Serial.printf("  Flow Rate: %.3f L/min\n", s_liveFlowLPM);
    Serial.printf("  Status:    %s\n", getFlowRangeStatusStr());
}
```

include/hardware_config.h

```
#ifndef HARDWARE_CONFIG_H
#define HARDWARE_CONFIG_H

#include <Arduino.h>

typedef enum {
    PUMP_DIR_FORWARD = 0,
    PUMP_DIR_REVERSE = 1
} PumpDirection;

#define IS_PIN_VALID(pin)           ((pin) >= 0)

#define PUMP1_L_PWM_PIN             32
#define PUMP1_R_PWM_PIN             33
#define PUMP1_L_EN_PIN              13
#define PUMP1_R_EN_PIN              26

#define PUMP1_EN_ACTIVE_HIGH        true
#define PUMP1_PWM_ACTIVE_HIGH       true

#define PUMP1_PWM_FREQ_HZ           5000
#define PUMP1_PWM_RESOLUTION_BITS   8
#define PUMP1_L_PWM_CHANNEL         0
#define PUMP1_R_PWM_CHANNEL         1

#define PUMP1_DEFAULT_DIRECTION     PUMP_DIR_FORWARD

#define PUMP2_L_PWM_PIN             18
#define PUMP2_R_PWM_PIN             19
#define PUMP2_L_EN_PIN              16
#define PUMP2_R_EN_PIN              17

#define PUMP2_EN_ACTIVE_HIGH        true
#define PUMP2_PWM_ACTIVE_HIGH       true

#define PUMP2_PWM_FREQ_HZ           5000
#define PUMP2_PWM_RESOLUTION_BITS   8
#define PUMP2_L_PWM_CHANNEL         2
#define PUMP2_R_PWM_CHANNEL         3

#define PUMP2_DEFAULT_DIRECTION     PUMP_DIR_FORWARD

#define CASE1_PUMP2_DEFAULT_NORMAL_DUTY 100

#define FLOAT_SWITCH_PIN            35
#define FLOAT_SWITCH_TRIGGERED_LEVEL LOW
#define FLOAT_SWITCH_DEBOUNCE_MS    30

#define FLOW_SENSOR_PIN                 14
#define FLOW_SENSOR_HZ_PER_LPM          98.0f
#define FLOW_SENSOR_UPDATE_INTERVAL_MS  1000
#define FLOW_SENSOR_MIN_RATED_LPM       0.3f
#define FLOW_SENSOR_MAX_RATED_LPM       6.0f

#define SYSTEM_SWITCH_PIN           -1
#define SYSTEM_SWITCH_ON_LEVEL      HIGH
#define SYSTEM_SWITCH_DEBOUNCE_MS   30

#define SOLENOID_VALVE_PIN          27
#define SOLENOID_ACTIVE_HIGH        true

#define PUMP3_L_PWM_PIN             23
#define PUMP3_R_PWM_PIN             25
#define PUMP3_L_EN_PIN              21
#define PUMP3_R_EN_PIN              22

#define PUMP3_EN_ACTIVE_HIGH        true
#define PUMP3_PWM_ACTIVE_HIGH       true

#define PUMP3_PWM_FREQ_HZ           5000
#define PUMP3_PWM_RESOLUTION_BITS   8
#define PUMP3_L_PWM_CHANNEL         4
#define PUMP3_R_PWM_CHANNEL         5

#define PUMP3_DEFAULT_DIRECTION     PUMP_DIR_FORWARD

// Circuit verified against schematic: External 10 kΩ pull-up resistor (R10)
    to 3.3V and 100 nF capacitor (C9) to GND.
// ESP32 GPIO 34 is input-only without internal pull-ups; MUST use
    pinMode(34, INPUT), never INPUT_PULLUP.
#define START_BUTTON_PIN            34
#define START_BUTTON_ACTIVE_LEVEL   LOW
#define START_BUTTON_DEBOUNCE_MS    30

#define FLUSH_BUTTON_PIN            START_BUTTON_PIN
#define FLUSH_BUTTON_ACTIVE_LEVEL   START_BUTTON_ACTIVE_LEVEL
#define FLUSH_BUTTON_DEBOUNCE_MS    START_BUTTON_DEBOUNCE_MS

#define CASE1_RETURN_DELAY_MS       10000UL
#define CASE2_FLUSH_DURATION_MS     90000UL
#define FLUSH_DURATION_MS           CASE2_FLUSH_DURATION_MS
#define CASE2_PUMP1_FLUSH_DUTY      50
#define CASE2_PUMP2_FLUSH_DUTY      100
#define CASE2_PUMP3_FLUSH_DUTY      50

#endif
```